The Journal of Open Source Software

DeepHyper: Massively Parallel Hyperparameter Optimization for Machine Learning

Romain Egele[1*¶], **Prasanna Balaprakash**[1*], **Gavin M. Wiggins**[1], and **Brett Eiffert**[1]

1 Oak Ridge National Laboratory, Tenesse, United States ¶ Corresponding author * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 10 March 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Machine learning models are increasingly applied across scientific disciplines, yet their effectiveness often hinges on heuristic decisions—such as data transformations, training strategies, and model architectures—that are not learned by the models themselves. Automating the selection of these heuristics and analyzing their sensitivity is crucial for building robust and efficient learning workflows. `DeepHyper` addresses this challenge by democratizing hyperparameter optimization, providing accessible tools to streamline and enhance machine learning workflows from a laptop to the largest supercomputer in the world. Building on top of hyperparameter optimization it unlock new capabilities around ensembles of models for improved accuray and uncertainty quantification. All of these organized around efficient parallel computing.

Figure 1: DeepHyper Logo

Statement of need

`DeepHyper` is a Python package for parallel hyperparameter optimization or neural architecture search. It provides access to a variety of asynchronous parallel black-box optimization algorithms `deephyper.search`. The software offers a variety of parallel programming backends such as Asyncio, threading, processes, Ray, and MPI `deehyper.evaluator`. The hyperparameter optimization can be single or multi-objective, composed of mixed variables, using

explicit or hidden constraints, and benefit from early-discarding strategies `deephyper.stopper`. Leveraging the results of hyperparameter optimization or neural architecture search it provides parallel ensemble algorithms `deephyper.ensemble` that can help improve accuracy or quantify disentangled predictive uncertainty.

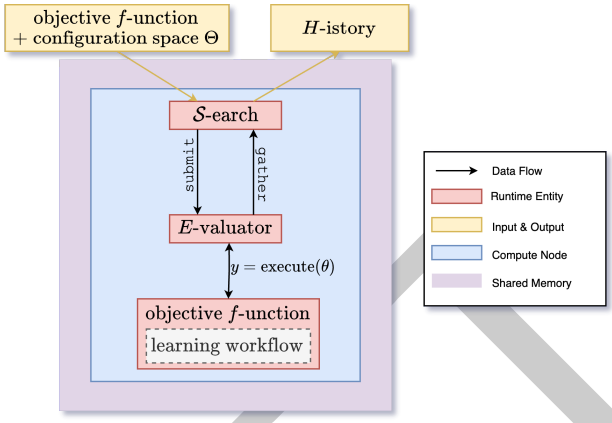


Figure 2: DeepHyper Software Architecture

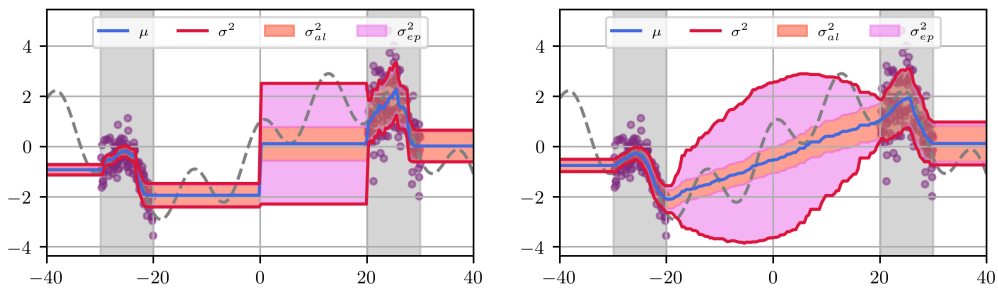
DeepHyper was designed to help research in the field of automated machine learning and also to be used out-of-the box in scientific projects where learning workflows are being developed.

Mathematics

DeepHyper main hyperparameter optimization algorithm is based on Bayesian optimization.

It can work with mixed-integer problems (i.e., including real, discrete, categorical values), with explicit or implicit constraints (e.g., $x_0 < x_1$, unexpected out-of-memory error), and stochastic objective functions $Y = f(x)$ where Y is a random variable, x is a vector of input hyperparameters and f is the objective function.

The Bayesian optimization of the DeepHyper relies by default on Extremely Randomized Forest assurrogate model to estimate $E_Y[Y|X = x]$. Extremely Randomized Forest are a kind of Random Forest where the split decision involves a random process for each newly created node of a tree. It provides smoother epistemic uncertainty estimates compared to usual Random Forests that use a deterministic "best" split decision.



Then, a custom acquisition function `UCBd`, focuses on the epistemic uncertainty of this surrogate for improved efficiency. It is also combined with a periodic exponential decay (impacting exploration-exploitation parameters of BO) to escape local solutions (Egele et al., 2023).

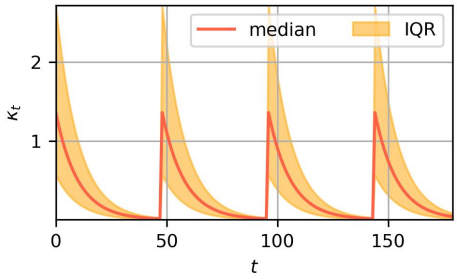


Figure 3: Periodic Exponential Decay for Bayesian Optimization

Batch parallel genetic algorithms are provided to resolve efficiently the sub-problem of optimizing the acquisition function and it is also more accurate than Monte-Carlo approches. An cheap and efficient multi-point acquisition strategy `qUCBd` is provided for better parallel scalability (Egele et al., 2023).

The multi-objective optimization is enabled by scalarization functions and objective rescaling (Egele and Chang et al., 2023).

The early-discarding strategies include asynchronous successive halving and a robust learning curve extrapolation (Egele et al., 2024).

The ensemble strategies is modular to allow: exploring models tested during hyperparameter optimization, classification and regression problems to be treated, disentangled uncertainty quantification (Egele et al., 2022).

Acknowledgements

We acknowledge contributions from Misha Salim, Romit Maulik, Venkat Vishwanath, Stefan Wild, Joceran Gouneau, Dipendra Jha, Kyle Felker, Matthieu Dorier, Felix Perez, Bethany Lush, Gavin M. Wiggins, Tyler H. Chang, Yixuan Sun, Shengli Jiang, rmjcs2020, Albert Lam, Taylor Childers, Z223I, Zachariah Carmichael, Hongyuan Liu, Sam Foreman, Akalanka Galappaththi, Brett Eiffert, Sun Haozhe, Sandeep Madireddy, Adrian Perez Dieguez, and Nesar Ramachandra. We also would like to thank Prof. Isabelle Guyon for her guidance on the machine learning methodologies developed in this software.

References

DRAFT